Segurança de Dados

Assignment 3: Stream Ciphers

Hugo Soares

8 de outubro de 2026

Questão 1

Ideia. Cifrar é fazer XOR com o keystream. Como XOR é reversível, decifrar é fazer XOR outra vez: $P = C \oplus K$.

```
def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def decrypt(ciphertext, keystream):
    return xor(ciphertext, keystream).decode()
```

Pseudo-OTP. O enunciado gera o keystream com a hora como seed:

```
random.seed(int(time.time()))
keystream = random.randbytes(n)
```

Isto é mau porque o módulo `random` é determinístico: quem adivinhar a hora reproduz o keystream todo. Com precisão de segundos, o espaço de procura é pequeno.

Ataque. O ciphertext tem 37 bytes, gerado a 15/09/2026 entre as 10:00 e as 11:00 UTC. Isso dá seeds de 1789466400 a 1789470000 (3601 tentativas). Para cada seed geramos 37 bytes e fazemos XOR. Guardamos o único resultado só com ASCII legível ($32 \leq b < 127$):

```
for seed in range(start, end + 1):
    ks = random.Random(seed).randbytes(len(ct))
    pt = xor(ct, ks)
    if all(32 <= b < 127 for b in pt):
        print(seed, pt.decode())
```

```
Q1 seed: 1789466511
Q1 msg: CHLORINE DOSING OVERRIDE CODE IS 7316
```

`os.urandom()` evita isto porque vai buscar aleatoriedade ao sistema operativo e não depende da hora.

Questão 2

Ideia. Os comandos são conhecidos: `OPEN` (4), `CLOSE` (5), `RESET` (5), `BYPASS` (6), `SHUTDOWN` (8). Para cada par com igual comprimento, $K = C \oplus P$. Mensagem curta usa só o prefixo do keystream.

Código. Primeiro achamos os keystreams completos com os ciphertexts de 8 bytes (só `SHUTDOWN` tem 8). Depois cada ciphertext é testado contra o prefixo de cada keystream:

```
keys = sorted({xor(c, p) for c in cts
               for p in cmds if len(c) == len(p) == 8})
for c in sorted(set(cts)):
    for k in keys:
        pt = xor(c, k[:len(c)])
        if pt in cmds:
            print(c.hex(), "->", pt.decode())
            break
```

Resultado. Há 4 keystreams (120 linhas no ficheiro):

Keystream	Hex
K_1	`8778e0399a6cf00a`
K_2	`a428809b98dd0ea9`
K_3	`d3c6299c0b2c7030`
K_4	`fb316b0987310073`

Exemplo com K_3: `9c966cd2` → OPEN, `81837ad95f` → RESET, `919f79dd587f` → BYPASS, `808e7cc84f63277e` → SHUTDOWN. Cada uma das 120 linhas bate num único keystream. Reutilizar keystream destrói o OTP.

Questão 3

Ideia. Seis mensagens com o mesmo K: $C_i \oplus C_j = P_i \oplus P_j$ — o K cancela. Só há maiúsculas ($65 \le b \le 90$), sem espaços, e C_3 tem 35 bytes.

Testar pequenas hipóteses. Para cada posição i, o byte $K[i]$ tem de dar letra em todos os 6 textos:

```
options = [[k for k in range(256)
  if all(65 <= (ct[pos] ^ k) <= 90 for ct in cts)]
  for pos in range(35)]
```

Isso deixa 8 a 18 candidatos por posição, o que por si só não decide nada. O que decide é testar palavras: se a palavra na posição p de C_3 implicar um $K[i]$ que não está em `options`, é rejeitada.

No início da frase, `QUE`, `NAO` e `UMA` são aberturas comuns. `QUE` daria a T_2 o início `E[M` e `NAO` daria a T_1 `T_K` (o `[` e o `_` são bytes que não são letra); ambos violam a regra. `UMA` passa:

```
T1: OSE   T2: ACI   T3: UMA
T4: ACI   T5: CLA   T6: OSA
```

É um ponto de partida útil, não a prova de que `UMA` é a única abertura possível.

Seguir "uma". A seguir a "uma" testam-se nomes na posição 3. `COISA` passa a regra das maiúsculas, mas dá `GUMAK` em T_1 e `FUIAA` em T_2, que não encaixam nos inícios já visíveis. `CHAVE` é natural num exercício sobre chaves reutilizadas e dá muito melhor:

```
T1: OSEGREDO   T2: ACIFRADE   T3: UMACHAVE
T4: ACIFRADE   T5: CLAUDESH   T6: OSATAQUE
```

Várias mensagens ganham palavras reconhecíveis ao mesmo tempo ("o segredo", "a cifra de", "os ataque…"). É apoio mais forte do que uma só frase plausível em C_3; não é preciso adivinhar nomes nem frases completas.

Descrever a chave. Na posição 8, `SECRETA` falha a regra das letras. `FORTE` passa, mas dá `PORMM` em T_1, o que produz `OSEGREDOPORMM` e não se lê naturalmente. Como o exercício é sobre reutilização de chaves, outra descrição razoável é `REUTILIZADA`:

```
T1: OSEGREDODEUMACIFRAD
T2: ACIFRADEVERNAMEAUNI
T3: UMACHAVEREUTILIZADA
T4: ACIFRADEVERNAMFOIPA
T5: CLAUDESHANNONDEMONS
T6: OSATAQUESDETEXTOCON
```

Isto completa frases inteiras nos outros textos ("o segredo de uma cifra", "os ataques de texto con…"). É por isso que se fica com `REUTILIZADA` em vez de `FORTE`. Continua a ser uma escolha linguística, não uma consequência da regra das maiúsculas.

Fechar a frase. Na posição 19, `NUNCA` falha a regra e `DEIXA` passa, estendendo T_1 a `OSEGREDODEUMACIFRADEVERE`, T_2 a `ACIFRADEVERNAMEAUNICAQUE` e T_6 a `OSATAQUESDETEXTOCONHECID`. Depois "uma chave reutilizada deixa" pede "de ser segura", testado em três partes: `DE` na posição 24, `SER` na 26 e `SEGURA` na 29.

Código.

```
def try_word(pos, word):
    ks = [c ^ p for c, p in zip(cts[2][pos:], word.encode())]
    if any(k not in options[pos + j] for j, k in enumerate(ks)):
        return None  # implicaria um byte que nao e letra
    return [bytes(ct[pos + j] ^ k for j, k in enumerate(ks)).decode()
            for ct in cts]

trials = [(0, "QUE"), (0, "NAO"), (0, "UMA"),
          (3, "COISA"), (3, "CHAVE"),
          (8, "SECRETA"), (8, "FORTE"), (8, "REUTILIZADA"),
          (19, "NUNCA"), (19, "DEIXA"),
          (24, "DE"), (26, "SER"), (29, "SEGURA")]

accepted = [(0, "UMA"), (3, "CHAVE"), (8, "REUTILIZADA"),
            (19, "DEIXA"), (24, "DE"), (26, "SER"), (29, "SEGURA")]
keystream = [None] * 35
for pos, word in accepted:
    assert try_word(pos, word) is not None
    for j, p in enumerate(word.encode()):
        keystream[pos + j] = cts[2][pos + j] ^ p
```

`options` guarda os bytes possíveis de K por posição. `try_word` calcula os bytes implicados pela hipótese e devolve `None` se algum falhar a regra; senão devolve o pedaço correspondente de cada texto claro, para comparar. `accepted` são as escolhas explicadas acima — não há escolha automática de palavras.

Resultado. As partes cobrem os 35 bytes de C_3 (as outras cinco continuam para lá disso):

```
T1: OSEGREDODEUMACIFRADEVERESIDIRAPENAS
T2: ACIFRADEVERNAMEAUNICAQUEOFERECESIGI
T3: UMACHAVEREUTILIZADADEIXADESERSEGURA
T4: ACIFRADEVERNAMFOIPATENTEADAEMMILNOV
T5: CLAUDESHANNONDEMONSTROUQUEOSIGILOPE
T6: OSATAQUESDETEXTOCONHECIDOEXPLORAMAR
```

O texto claro completo de C_3 é `UMACHAVEREUTILIZADADEIXADESERSEGURA` ("Uma chave reutilizada deixa de ser segura"). O K de 35 bytes é idêntico a partir dos 6 textos, o que confirma a decifra.

Questão 4

Ideia. $C = P \oplus K$. Para trocar P por P' sem saber K, basta anular P e pôr P': $C' = C \oplus P \oplus P'$.

```
c = bytes.fromhex("caffea...0071a")  # 30 bytes
p = b"PLC=07;CMD=SET;NAOH_PPM=00100;"
pp = b"PLC=07;CMD=SET;NAOH_PPM=11100;"
print(xor(c, xor(p, pp)).hex())
```

```
Q4 forged: caffea70ef5b03e9f898bebf13e125a8b81e1a75aea1f20e166abbc0071a
```

Só mudam os bytes 24 e 25 (`17 6b` → `16 6a`), que são os dois primeiros dígitos de `00100` → `11100`. O ChaCha20 não foi quebrado: o problema é a falta de autenticação. Sem MAC, virar bits no ciphertext vira os mesmos bits no plaintext. Com ChaCha20-Poly1305 a forja seria detetada.

Questão 5

Ideia. Mesmo nonce + mesma chave = mesmo keystream K. Como conhecemos P_1 (heartbeat de 50 bytes), $K = C_1 \oplus P_1$. Depois $P_2 = C_2 \oplus K$.

```
p1 = b"HEARTBEAT;PLC=07;STATUS=OK;NAOH_PPM=00100;PH=07.4;"
k = xor(c1, p1)
print(xor(c2, k[:len(c2)]).decode())
```

```
Q5 msg: PLC=07;CMD=SETPWD;PWD=Tr3at!Pl4nt#26;
```

P_2 tem 37 bytes e o heartbeat tem 50, por isso recupera-se tudo. Se P_2 fosse maior que 50 bytes, só os primeiros 50 se recuperavam; o resto precisaria de mais plaintext conhecido.

Conclusão

Keystream previsível, reutilizado ou sem autenticação chega para atacar, sem quebrar o algoritmo. Usar gerador seguro, nonce único e modo autenticado (ChaCha20-Poly1305).